

Playwright Cmds & all

07 September 2025 10:59

Definitions & Notes

- Workers are individual new sessions, all running parallelly.
- Location Strategies Order (& How to find in inspect (Shortcuts))-**
 - Id - #
 - Name
 - Class
 - CSS - [] Copy paste the required attribute
 - Xpath
- Plugins for finding locator:**
 - SelectorsHub
 - LetXPath
- Assertions**
 - Visible/Hidden
 - Present/Not Present - Element is present in the DOM or not
 - Enabled/Disabled
 - Text Match / Mismatch
 - Element Attributes - It checks whether the element has the expected attributes or not
 - URL
 - Full URL Assertion
 - Partial URL Assertion
 - Title
 - Full Title Assertion
 - Partial Title Assertion
 - Screenshot - It checks the look and feel of the elements on the website
Procedure:
 - When the screenshot assertion runs for the first time, it will take up a screenshot of this entire window and save it that image in the test folder. Hence for the first time it is expected to fail because it doesn't have any screenshot to compare for.
 - Next time when the test runs, it will compare the web screen with the screenshot, which it took earlier.
 - If there is any slight change in an element's alignment, then the screenshot assertion will notify it.
- Different UI Elements available**
 - Simple UI Elements:**
 - Text box
 - Button
 - Radio Buttons
 - Check Boxes
 - Single Static Dropdown
 - Multi Static Dropdown
 - Dynamic Dropdown
 - Simple Alert
 - Confirmation Alert
 - Alert with Textbox
 - Advanced UI Elements:**
 - Web Table
 - Frames
 - Tab/Window Handling
 - Calendar
 - Drag and Drop
 - Downloading File
 - Uploading File
- Difference between single static drop down and multi static drop down**
 - In a single static drop down at a time only one option can be selected whereas in a multi static drop down at a time multiple options can be selected
- Dynamic dropdown 2 handling ways -**
 - Searchable** - Required option is 1st typed in the textbox of dynamic dropdown and when it is visible in the options list it is selected
 - Non Searchable** - Required option is searched in the dropdown list and is selected
- In handling the popup boxes -**

`page.on` is an event listener, so:

- The event that triggers it should be written below it in both JS and TS.
- It cannot close the event so the event should be closed using

```
await dialog.accept(); or
await dialog.dismiss();
```

Commands & Syntax:

- To run a specific test case in a file: `npx playwright test -g '<testCaseName>'`
- To specific run in a specific browser (via the terminal): `npx playwright test --project=<browserName>`
- To run the files on headless (Via Config file): In use parameter: `headless:true`

Extra ways to locate:

- `await page.locator('[name="user-name"]');` - Directly using object property
- `await page.locator('text=Login').click();` - Using Text way 1
- `await page.locator('input:has-text("Login")').click();` - Using Text way 2

To pause the test execution:

```
await page.pause();
```

Assertions:

Syntax:

```
await expect(<locator strategy>).to<methodname>(<arguments>)
```

1.

a. Visible:

```
.toBeVisible()
```

b. Hidden:

```
.toBeHidden()
```

2.

a. Present:

```
.toHaveCount(<number>) - this line checks that an element located is present mentioned <number> of times in the DOM
```

b. Not Present:

```
.not().toHaveCount(1) - this line checks that not a single element located is present in the DOM
```

3.

a. Enabled:

```
.toBeEnabled()
```

b. Disabled:

```
.toBeDisabled()
```

4.

a. Text Match:

```
.toHaveText('<Text>') - this line checks whether the given text is present in the located element or not
```

b. Text Mismatch:

```
.not().toHaveText('<Text>') - this line checks whether the given text is not present in the located element or not
```

5. Element Attributes:

For any attribute -

```
.toHaveAttribute('<attributeName>', '<attributeValue>')
```

For class attribute -

```
.toHaveAttribute('class', /.*<classname>/) - The <classname> value should be written in the regular expression - /* /
```

6. URL:

a. Full URL assertion:

```
.toHaveURL('<Full URL>')
```

b. Partial URL assertion:

```
.toHaveURL(/<Partial part of URL>/) - The partial part of the URL should be written in the regular expression //
```

7. Title:

a. Full Title assertion:

```
.toHaveTitle('<title>')
```

b. Partial Title assertion:

```
.toHaveTitle(/.*<Partial part of Title>/) - The partial part of title should be written in the regular expression - /* /
```

8. Screenshot:

```
await expect(page).toHaveScreenshot();
```

Radio button & Checkbox assertion:

1. Assertion way 1

```
expect(maleRadioButton).not.toBeChecked();
```

```
await maleRadioButton.check();
```

- It cannot close the event so the event should be closed using


```
await dialog.accept(); or
await dialog.dismiss();
```
- In prompt popup, the text can be entered using


```
await dialog.accept('<text>'); statement.
```
- **Difference between web table and pagination table**
 - Web table is the normal HTML table whereas a pagination table is the one in which multiple pages of tables are present.
- **For repeating tasks -**

```
async function name(param:type){<code>}
```

Can be made & called out in the test cases using **await** keyword.
THE FUNCTION SHOULD BE WRITTEN OUTSIDE THE TEST CASES.
- **For selecting items in a table - .filter({<code>})** can be used.
- **Different types of frames and their handling:**
 - **Why their handling procedure is needed:** As the elements inside the frames cannot be located directly from the page.
 - **2 Types:**
 1. **Frame:** Frames are an HTML feature used to divide a web page into multiple independent sections, each with its own source URL. Each frame acts as a separate document with its own HTML, CSS, and JavaScript.
 2. **Iframe (i.e. Inner Frame):** A HTML element that allows you to embed another HTML document within the current document. It creates a nested browsing context and displays the content of the specified source URL.
 - **Both are handled in the same way - 3 ways:**
 1. By frame locator method
 2. By frame with name argument
 3. By frame with URL argument
- **Calendar types:**
 1. Fillable calendar
 2. Date pickable calendar
- **For handling dates & times - JS Dependency: Luxon:**
 - Luxon is a library for working with the dates and times in JavaScript.
 - This also can be used for TypeScript project also.
- **Drag and drop 2 ways:**
 1. Combination of hover and mouse up & down commands
 2. dragTo command
- **File downloading:**
 - To handle the file downloading process, an event listener is used the same way as used for the window handling.
 - Once the playwright finishes the test, it will immediately delete the file which it downloads.
 - **To save the file - 2 ways:**
 1. Using the suggested file name
 2. Using a customized file name
- **File Uploading - 2 ways:**
 1. Using promise and setFiles command line
 2. Using setInputFiles command line

1. Assertion way 1

```
expect(maleRadioButton).not.toBeChecked();
```

```
await maleRadioButton.check();
expect(maleRadioButton).toBeChecked();
```

2. Assertion way 2

```
expect(await femaleRadioButton.isChecked()).toBeFalse();
```

```
await femaleRadioButton.check();
expect(await femaleRadioButton.isChecked()).toBeTruthy();
```

- **.toBeChecked()** - To check whether an element is checked; adding **.not** before the method makes it to check whether the element is not clicked.
- **.toBeFalse()** - Checks if the output of the action to which it is attached gives false output
- **.toBeTruthy()** - Checks if the output of the action to which it is attached gives true output

- **Equality assertion:**
.toEqual('<text>')

• To execute a specific single test case:

```
test.only()
```

• Methods on Text box & Button

- **.pressSequentially('<userInput>', {delay:200})** - Gives the typing action; the optional argument **delay: <number>** executes the action as slowly as mentioned in it
- **.press('<keyValue>')** - Presses the mentioned key
- **.click({button: '<argument>'})** - Presses specific key; Arguments available are - left, middle, right

• Method to select options from a single static dropdown:

.selectOption('<id of the dropdown>', {attribute name: '<attribute value>'}) - this is available only for the elements with <select> tag.

- 3 ways:

1. By value attribute:

```
.selectOption('<id of the dropdown>', {value: '<value>'})
```

2. By label attribute:

```
.selectOption('<id of the dropdown>', {label: '<value>'})
```

3. By index attribute:

```
.selectOption('<id of the dropdown>', {index: <value>})
```

• Method to select options from a multi static dropdown: Same as single static dropdown but multiple options can be passed in these {} as an array.

```
.selectOption('<id of the dropdown>', [
  {value: '<value>'},
  {label: '<value>'},
  {index: <value>}
]);
```

• Searchable Dynamic dropdown code:

```
test("Searchable Dynamic Dropdown Handling", async({page})=>{
  await page.goto('https://demo.automationtesting.in/Register.html');
  await page.locator('span[role="combobox"]').click();
  await page.locator('input[role="textbox"]').fill('India');
  await page.locator('ul#select2-country-results>li').click();
  await page.waitForTimeout(2000);
  await page.close();
});
```

• Non Searchable Dynamic dropdown code:

```
test("Non Searchable Dynamic Dropdown Handling", async({page})=>{
  await page.goto('https://demo.automationtesting.in/Register.html');
  await page.locator('span[role="combobox"]').click();
  await page.locator('ul#select2-country-results').locator('li', {hasText: 'India'}).click();
  await page.waitForTimeout(2000);
  await page.close();
});
```

• Selecting single checkbox in the table code:

```
test("Selecting Single CheckBox in the Table", async({page})=>{
  await page.goto('https://testautomationpractice.blogspot.com/');
  const table = page.locator('#productTable');
  const columns = table.locator('thead tr th');
  const rows = table.locator('tbody tr');
  //Find the required row -> Check the checkbox
  const matchRow = rows.filter({
    has: page.locator('td'),
    hasText: 'Tablet' //indicates td having text 'Tablet' should be
  });
  await matchRow.locator('input').check();
  await page.waitForTimeout(2000);
  await page.close();
});
```

```
}}
```

- **Selecting multiple checkboxes in the table code:**

```
test("Selecting Multiple CheckBoxes in the Table", async({page})=>{
  await page.goto('https://testautomationpractice.blogspot.com/');
  const table = page.locator('#productTable');
  const columns = table.locator('thead tr th');
  const rows = table.locator('tbody tr');
  await selectProduct(rows, page, 'Smartphone');
  await selectProduct(rows, page, 'Laptop');
  await selectProduct(rows, page, 'Tablet');
  await page.waitForTimeout(2000);

  await page.close();
})
async function selectProduct(rows:any, page:any, productName:any) {
  const matchRow = rows.filter({
    has: page.locator('td'),
    hasText: productName //indicates td having text productName should
  })
  be filtered
  })
  await matchRow.locator('input').check();
}
```

- **Handling content of pagination tables (Single page & all pages):**

```
test("Printing all items from Page1 of pagination table", async({page})=>{
  await page.goto('https://testautomationpractice.blogspot.com/');
  const table = page.locator('#productTable');
  const columns = table.locator('thead tr th');
  const rows = table.locator('tbody tr');
  for (let i=0; i<await rows.count(); i++){
    const row = rows.nth(i); //i th tr
    const rowData = rows.locator('td'); //tr -> td
    for(let j=0; j<await rowData.count(); j++){
      const cellContent = await rowData.nth(j);
      console.log(cellContent);
    }
  }

  await page.waitForTimeout(2000);

  await page.close();
})
test("Printing all items from all pages of pagination table", async({page})=>{
  await page.goto('https://testautomationpractice.blogspot.com/');
  const table = page.locator('#productTable');
  const columns = table.locator('thead tr th');
  const rows = table.locator('tbody tr');
  const pages = page.locator('#pagination li a');
  const totalPages = await pages.count();
  console.log("Total No. of Pages:" +totalPages);
  for (let p=0; p<totalPages; p++){
    if(p>0){
      await pages.nth(p).click();
    }
  }
  for (let i=0; i<await rows.count(); i++){
    const row = rows.nth(i); //i th tr
    const rowData = rows.locator('td'); //tr -> td
    for(let j=0; j<await rowData.count(); j++){
      const cellContent = await rowData.nth(j);
      console.log(cellContent);
    }
  }
  await page.waitForTimeout(2000);

  await page.close();
})
```

- **Frame Handling methods (excluding .frameLocator method - 2 more methods):**

1. `.frame({url:'<URL>'})`
2. `.frame({name:'<nameValue>'})`

- **Other frame related methods:**

- To get the array of all the frames:
`.frames()`
- To get all the nested frames:
`.childFrames()`

- **To count no. of contexts & pages:**

```
.context().pages()
```

- **Calendar selection using fill method code:**

```
test("Calendar using fill method", async({page})=>{
  await page.goto('https://www.lambdatest.com/selenium-playground/bootstrap-date-picker-demo');
  let date = "1993-10-18"; //Format is acc. to the locator's value
  await page.locator('#birthday').fill(date);
  await page.waitForTimeout(2000);
})
```

- **Installation of Luxon dependency: npm install -D luxon**

- **Drag & drop:**

1. `await <source>.hover();`
`await page.mouse.down();` - Continuously press down on the element
 Above 2 commands makes the source element to be continuously selected and being drag around

`await <target>.hover();`
`await page.mouse.up();` - Release of mouse action
 Causes the source element to be released in the target area - drop

2. `await <source>.dragTo(<target>);`

- **File downloading code:**

```
test('Downloading a File', async ({page}) => {
  await page.goto('https://demo.automationtesting.in/FileDownload.html');
  await page.locator('#textbox').click();
  await page.locator('#textbox').pressSequentially('Playwright Udem Course');
  await page.locator('#createTxt').click();
  const [download] = await Promise.all([
    page.waitForEvent('download'),
    page.locator('#link-to-download').click()
  ])
  const path = await download.path();
  console.log("Downloaded path" + path);
  //Way - 1 Using default filename
  const fileName = download.suggestedFilename();
  await download.saveAs(fileName);
  //Way - 2 Using customised filename
  const customisedfileName = "Playwright_Downloaded_File";
  await download.saveAs(customisedfileName);
})
```

- **File uploading code:**

1. `test('File uploading - Way 1', async ({page}) => {`
`await page.goto('https://blueimp.github.io/jQuery-File-Upload/');`
`const [uploadFile] = await Promise.all([`
`page.waitForEvent('filechooser'),`
`page.locator('input[name="files[]"]').click()`
`])`
`await uploadFile.setFiles(['C:/Users/2430147/OneDrive -`
`Cognizant/Documents/Test/tests/test.docx', 'C:/Users/2430147/OneDrive -`
`Cognizant/Documents/Test/tests/test.pdf']);`
`await page.waitForTimeout(2000);`
`})`
 - `.waitForEvent('filechooser')` - file chooser means, as the name implies, it will choose the file for us from the explorer
 - `.setFiles(['<filename1>', '<filename2>'])` - takes multiple files as an array; needs `Promise.all` for the uploading.
2. `test('File uploading - Way 2', async ({page}) => {`
`await page.goto('https://blueimp.github.io/jQuery-File-Upload/');`
`await page.setInputFiles('input[name="files[]"]', 'C:/Users/2430147/OneDrive -`
`Cognizant/Documents/Test/tests/test.pdf');`
`await page.waitForTimeout(2000);`
`})`
 - `.setInputFiles('<locator>', '<filename>')` - Doesn't need promise for its execution; directly uploads the file to the locator.